Preparing multiplex immunofluorescence data in QuPath for QXYCell

QuPath 0.7 workflow for platform-independent multiplex immunofluorescence data

Applies to: QuPath 0.7.0 and QXYCell 0.1

Purpose: create the measurement, annotation, segmentation, and classifier assets that QXYCell can validate and import.

> QXYCell is platform-independent. It does not require COMET data or COMET-specific background subtraction. A multichannel OME-TIFF can be used when QuPath reads it correctly and the checks below are satisfied. Image correction, registration, unmixing, background removal, and marker QC remain the responsibility of the acquisition or preprocessing workflow.

What QXYCell needs

The only unconditional input is a QuPath cell measurement table. Other assets in the QuPath project folder enable specific QXYCell features.

Asset	Needed for	QXYCell requirement
Cell measurements (`.csv` or `.tsv`)	Every run	Filename contains `measurement`, or is exactly `detections.csv`/`detections.tsv`
Annotation GeoJSON	Sample/region labels and exclusions	One file per image; filename stem must match the measurement table's `Image` value
Cell GeoJSON	Cell boundary geometry	Contains QuPath cell objects and their Object IDs
Single-measurement classifier JSON	Creating a threshold table	Simple classifier JSONs saved anywhere below the QuPath project folder
Filled threshold table	Applying marker positivity	Reviewed TSV/CSV with per-image threshold values

Keep every exported input inside the QuPath project folder and pass that one folder to QXYCell. QXYCell discovers files recursively, so the assets may be arranged in subfolders such as `qxycell_input`. Do not create a second input directory. Keep generated QXYCell output folders beside the project folder, not inside it.

1. Verify the image and pixel size

Open each project image in QuPath and confirm that it is the intended fluorescence image or series, that the expected marker channels are present, and that there is no obvious channel misregistration.

In the Image tab, check the pixel calibration before analysis:

- Pixel width and pixel height are reported in micrometres (`µm`).
- The values are plausible for the microscope and acquisition settings.
- Pixel width equals pixel height. QXYCell currently supports square pixels only.
- Record the verified pixel size with the analysis.

QuPath reads pixel calibration from image metadata when possible, but the metadata can be absent or wrong. Pixel size is the important physical scale; nominal objective magnification is not a substitute.

Choose the QXYCell pixel size

QXYCell's default is 0.28 µm/pixel. Use the default only when the verified QuPath pixel size is 0.28 µm in both directions.

Import the cell measurements first, then pass the verified pixel size when adding annotation and cell GeoJSON geometry:

```
adata = qxy.import_cells("/path/to/qupath_project")
qxy.add_annotations(adata, pixel_size_um=0.325)
```

The terminal command performs Stage 1 only:

```
qxycell import-cells /path/to/qupath_project
```

The value must be one positive, finite number. Do not average unequal pixel width and height values. Correct or re-export an anisotropic image before using this QXYCell workflow.

Why this matters: QuPath's exported centroid columns are already in micrometres, whereas QuPath GeoJSON geometry is expressed in full-resolution pixel coordinates. QXYCell uses `pixel_size_um` to align annotation and cell polygons with the centroids.

2. Create a QuPath project and add images

1. Create a new, empty folder dedicated to the QuPath project. This is the QuPath project folder that you will later pass to QXYCell.
2. In QuPath, choose File > Project… > Create new project and select the empty folder.
3. Add the source images by dragging them into QuPath or choosing File > Project… > Add images.
4. If a file contains multiple images or series, enable the image selector and import only the intended series.
5. Set the image type to fluorescence where required.
6. Open every project entry and verify the image and pixel size as described above.
7. Save the project.

QuPath stores links to the source images. If images are moved, use File > Project… > Check project URIs to repair their locations.

3. Create analysis annotations

Annotations define regions used for segmentation and downstream grouping.

1. Select a drawing tool such as Rectangle, Ellipse, Polygon, Brush, or Wand.
2. Draw the required analysis regions on the image.
3. Assign every exported annotation a meaningful QuPath classification or name.
4. Avoid overlapping annotations that represent mutually exclusive samples.
5. Save the image data.

QXYCell annotation conventions

- Include the word `sample` anywhere in each sample annotation name. Matching is case-insensitive. For example, `Sample_01`, `sample_tumour`, and `PatientA Sample` are automatically imported into `adata.obs["Sample"]`; the complete annotation name becomes the sample value.
- Use unique sample annotation names and avoid overlaps. Cells inside more than one sample annotation are labelled `Ambiguous` and reported as conflicts.
- Give every region that should later be excluded a common word in its name. For example, name regions `Ignore_fold`, `Ignore_edge`, and `Ignore_staining`, then remove their cells with `qxy.remove_cells(adata, remove_cells="ignore")`. The matching word can be changed, but the same word must be used consistently in the annotation names and the `remove_cells=` argument.
- Other annotation labels become boolean columns named `annotation__<safe_label>` in `adata.obs`.
- QuPath TMA core assignments are not imported from annotation GeoJSON. Create the TMA grid in QuPath before cell detection and measurement export. QuPath then includes each cell's grid assignment in the measurement table column named exactly `TMA Core`, which QXYCell converts to `CoreID`.

4. Segment cells with InstanSeg

InstanSeg is the recommended workflow represented here, but QXYCell does not require a particular segmentation algorithm. It requires QuPath cell objects with measurements and stable Object IDs.

Install and open InstanSeg

1. Choose Extensions > Manage extensions.
2. Install the Deep Java Library and InstanSeg extensions if they are not already installed.
3. Restart QuPath when requested.
4. Open Extensions > InstanSeg.

Test before running the complete region

1. Create or select a small representative test annotation.
2. Choose a fluorescence model appropriate for the available channels and desired output.
3. Select the correct input channels. Do not assume every channel improves segmentation.
4. Set the output to cells when cell boundaries are required.
5. Enable Make measurements.
6. Run the model on the test annotation.
7. Inspect nuclear/cell boundaries in dim, bright, crowded, sparse, tissue-edge, and artefact regions.
8. Adjust model, channels, tile padding, or other parameters when boundaries are systematically wrong.
9. Record the model and parameters, then run the accepted settings on the complete analysis annotation.

Use the verified physical calibration: InstanSeg models operate at a physical resolution and QuPath uses pixel calibration when rescaling image data. A successful run is not proof that segmentation is biologically accurate; visual review remains required.

5. Review cell measurements

Confirm cell measurements

Open Measure > Show detection measurements and confirm that the cell rows contain:

- `Object ID`
- `Centroid X µm`
- `Centroid Y µm`
- the intended cell, cytoplasm, or nucleus marker measurements
- `TMA Core` when a QuPath TMA grid is used and core identity is required

QXYCell imports marker columns whose names contain `mean` or `median` (case-insensitive). Confirm that the intended marker measurements use one of those summaries.

6. Export the cell measurement table

1. Save all open QuPath image data first.
2. Choose Measure > Export measurements.
3. Move the required project images into the selected list.
4. Set Export type to cells.
5. Export all columns unless you have verified a restricted column list includes every required identity, coordinate, and marker measurement.
6. Choose comma-separated (`.csv`) or tab-separated (`.tsv`) output.
7. Name the file `measurements.csv` or `measurements.tsv`.
8. Save it below the QuPath project folder, preferably in a subfolder named `qxycell_input`.

Required columns are exactly:

```
Image
Object ID
Centroid X µm
Centroid Y µm
```

Include TMA core identity in the measurement export

For a tissue microarray project:

1. Create and label the grid with the commands under TMA.
2. Confirm that detected cells are associated with the intended cores.
3. In Measure > Export measurements, keep Export type set to cells and choose a tab separator.
4. Include the exact column `TMA Core` and export `measurements.tsv`.
5. Check the exported header before running QXYCell.

`qxy.import_cells()` preserves `TMA Core` and automatically creates the categorical `CoreID` column. If `TMA Core` is absent, QXYCell does not create `CoreID`.

QuPath documents the TMA grid commands and the project measurement exporter.

One table may contain cells from multiple images. Large tables may contain millions of rows; do not resave them from spreadsheet software that may truncate rows, alter identifiers, or change headers.

7. Create optional marker thresholds

Marker thresholds are separate from the cell measurement table, but they must be created after cells have been segmented so the classifier can be previewed and reviewed on cell measurements.

For each marker that requires a QuPath-derived starting threshold:

1. Choose Classify > Object classification > Create single measurement classifier.
2. Filter to cells.
3. Select one marker measurement, preferably a reviewed mean or median measurement from the appropriate compartment.
4. Set the below/above-threshold classes and enable live preview.
5. Review positive and negative cells across representative images and tissue conditions.
6. Save the classifier with a short, unique marker-based name.

QXYCell reads simple single-measurement classifier JSONs. Composite or malformed classifiers are reported but are not converted into threshold rows. Classifier thresholds are starting definitions: generate and review the QXYCell threshold table before applying positivity.

8. Export annotation GeoJSON

Repeat for every image that has annotations to import.

1. Open the image and select only the annotation objects to export.
2. Choose File > Object data… > Export as GeoJSON. In some QuPath 0.7 installations the command may appear as File > Export objects as GeoJSON.
3. Export selected objects as a GeoJSON `FeatureCollection`.
4. Exclude measurements to keep the file smaller.
5. Use no compression and the `.geojson` extension.
6. Name the file from the image name exactly, removing only `.ome` and the image extension.
7. Save it below the QuPath project folder, preferably in `qxycell_input/annotations`.

Examples:

Measurement `Image` value	Annotation filename
`slide01.ome.tif`	`slide01.geojson`

Measurement `Image` value	Annotation filename
`region_A.ome.tiff`	`region_A.geojson`
`sample-3.tif`	`sample-3.geojson`

QXYCell matches annotation geometry to image rows by this stem. A file named `slide01-annotations.geojson` will not match `slide01.ome.tif`.

9. Export cell GeoJSON

Cell GeoJSON is optional but required for `cell_polygon_wkt` and cell-boundary plots.

1. Select the cell detection objects for one image, not the analysis annotations.
2. Choose the GeoJSON export command.
3. Export selected objects as a `FeatureCollection`.
4. Exclude measurements; the separate measurement table is the data source.
5. Preserve QuPath Object IDs.
6. Save it below the QuPath project folder, preferably in `qxycell_input/cells`, as `<image-stem>-cells.geojson`, for example `slide01-cells.geojson`.

QXYCell matches cell polygons to measurement rows by QuPath Object ID. Do not refresh Object IDs between the measurement and cell-GeoJSON exports.

For very large images, cell GeoJSON can be large. Export one image at a time and verify that the feature count is plausible.

10. Organize the QuPath project folder

Recommended layout:

```
qupath_project/
|-- project.qpproj
|-- data/
|-- classifiers/
|   `-- object_classifiers/
|       |-- CD3.json
|       `-- PanCK.json
`-- qxycell_input/
    |-- measurements.tsv
    |-- annotations/
    |   |-- slide01.geojson
    |   `-- slide02.geojson
    |-- cells/
    |   |-- slide01-cells.geojson
    |   `-- slide02-cells.geojson
    `-- thresholds/
        `-- thresholds.tsv
```

The source OME-TIFF files do not have to be copied into the QuPath project folder when the project contains valid links to their existing locations. Measurements, GeoJSON, classifiers, and project-level threshold tables must remain inside the project folder so QXYCell has one location to search for inputs.

11. Run the QXYCell preflight

```
import qxycell as qxy

project_dir = "/path/to/qupath_project"
report = qxy.check(project_dir, count_rows=True)

print(report.ok)
print(report.n_errors, report.n_warnings)
```

Or from a terminal:

```
qxycell check /path/to/qupath_project --count-rows
```

Review `check_report.txt`, `check_report.json`, and the tables in the generated sibling check folder. Resolve errors before import. Review warnings rather than assuming they are harmless.

The QXYCell check validates exported assets; it cannot confirm microscope calibration, channel identity, registration, biological staining quality, or segmentation accuracy. Those remain explicit QuPath pre-verification points.

12. Import cells and add annotations

Import the cell measurements to create the base AnnData checkpoint:

```
adata = qxy.import_cells(project_dir)
```

Then import GeoJSON geometry using the recorded pixel size. Use `0.28` only for verified 0.28 µm square pixels:

```
qxy.add_annotations(adata, pixel_size_um=0.325)
```

After import, confirm the stored audit value:

```
adata.uns["qxycell"]["pixel_size_um"]
```

Plot cell centroids and annotation or cell boundaries as a final alignment check before downstream analysis.

Final handoff checklist

- [ ] QuPath 0.7.0 project saved for every image.
- [ ] Image series, dimensions, channels, and physical units verified.
- [ ] Pixel width and height are equal and the scalar value is recorded.
- [ ] Segmentation visually reviewed across representative regions.
- [ ] `measurements.csv` or `measurements.tsv` contains all required columns.
- [ ] TMA projects include the exact `TMA Core` column in the cell measurement export.
- [ ] Annotation GeoJSON filenames match their image stems exactly.
- [ ] Cell GeoJSON retains the Object IDs used in the measurement export.
- [ ] Classifier JSONs are simple single-measurement classifiers where used.
- [ ] `qxy.check()` errors resolved and warnings reviewed.
- [ ] `qxy.add_annotations()` uses the verified `pixel_size_um` value.
- [ ] Spatial overlay alignment reviewed after import.

Troubleshooting

No measurement files found

Rename the export so its filename contains `measurement`, or use `detections.csv`/`detections.tsv`.

Missing required columns

Re-export cells and include `Image`, `Object ID`, `Centroid X µm`, and `Centroid Y µm` without editing the headers.

Annotations are discovered but not assigned

Check that each annotation filename stem matches the corresponding `Image` value after removing `.ome` and the image extension. Confirm that the QXYCell pixel size matches QuPath.

Cell polygons are missing

Confirm that cell objects, rather than annotations, were exported and that QuPath Object IDs match the measurement table.

Polygons are offset or scaled incorrectly

Stop downstream analysis. Recheck QuPath pixel calibration and the value passed as `pixel_size_um`; then rerun the import.

Pixel width and height differ

This QXYCell workflow does not support non-square pixels. Do not average the values. Correct or resample the image upstream and repeat segmentation and export.

Official references

- QuPath 0.7.0 releases
- QuPath projects
- QuPath image concepts and pixel calibration
- QuPath InstanSeg workflow
- QuPath measurement export
- QuPath GeoJSON export
- QuPath 0.7.0 command reference